AI usage

AI useage

This file is for documenting all ai usage :D

Created by claude: doc/markdown/style/rats.css doc/markdown/style/template.md prompt: use the RAT logo in assets and make a cool markdown theme that is mainly it and about rats

ai question -> css template or both? -> both

Created by claude: Readme.md prompt: can you write the README.md for github also with the style if possible idk if that works on github

ai question -> has RAT a meaning? -> Remote Access Topology -> what does it do? -> Network topology with users that have diffrent privileges diffrent logins like ssh

can be added to devices to change settings remote see also the Project_Planning.md file

Edited by claude: doc/assets/drawio/DB_Sketches.drawio prompt: we have done the 3 Normalform RM Diagramm and ERM Diagramm can you add us the second and first Normalform of the Database

========= PERMISSION SYSTEM =========

Model: Claude (claude-opus-4-8) via Claude Code Date: 2026-06-16

Overall user prompt (translated/summarized): “NetworkObjectPermission has an integer `permissions`: 0 = Hidden (user may not see it; same as having no permission row at all) 1 = See (user can see the device + interfaces; may add their own logins/snmp settings) 2 = Edit (may add/edit/delete interfaces and change NO settings like name; cannot delete the NO) 3 = Admin (may grant/change roles of users with LOWER rights than himself, i.e. 0..2) 4 = Owner (may change Admins, grant/remove Owner, and delete the object) Add the permission checks to every route so not everyone can do everything, only users with the right level. Also document all AI usage (model, prompt, …) and mark every changed spot in the code with `#KI Claude + <prompt number>`. While reading the code, note possible problems with `#KI Claude detected problem why/what:`.”

How the prompt numbers map to the code markers (search for `KI Claude <KI-N>`):

src/permissions.py (NEW FILE) Created a central helper module with the enum constants (HIDDEN/SEE/EDIT/ADMIN/OWNER) plus: - get_permission_level(db, user, network_object_id) -> effective level; no row == HIDDEN; global is_admin == OWNER - require_permission(db, user, network_object_id, min_level) -> raises 404 if Hidden (so existence stays secret), 403 if too low.

src/routers/networkObject.py - GET / : only returns NOs where the user has See(1)+ (admins: all) - POST / : requires is_admin or user.canCreate; creator is made Owner of the new object (otherwise nobody could see it) - PUT /{id} : requires Edit(2)+ - DELETE /{id} : requires Owner(4); also cleans up permission rows. Removed the bogus request body from DELETE.

src/routers/networkObjectInterface.py - GET / : only interfaces of NOs the user may See(1)+ - POST / : Edit(2)+ on the target NO - PUT /{id} : Edit(2)+ on both old and new NO of the interface - DELETE /{id} : Edit(2)+ on the interface's NO

src/routers/networkObjectConnection.py - GET / : only connections touching a See(1)+ NO - POST / : Edit(2)+ on the NOs of both endpoint interfaces - PUT /{id} : Edit(2)+ on the NOs of all attached interfaces - DELETE /{id} : same; also fixed wrong-model bug (see problems below)

src/routers/networkObjectPermission.py - Added `target_user_id` to the input (you must say WHO you grant to). - assert_can_grant() enforces: * only Admin(3)/Owner(4) may grant * Admin may only touch users with strictly lower rights, and may only assign levels 0..2 * Owner may assign anything incl. Admin/Owner - GET / : own rows + (for Admin/Owner) all rows on managed objects - POST / : grant/update with the rules above; upserts instead of creating duplicate (user, object) rows - PUT /{id} : same rules; forbids retargeting the row to another object/user - DELETE /{id} : treated as "set to Hidden(0)", same rules apply

—— Problems Claude detected (marked in code with `#KI Claude detected problem why/what:`) ——

1. networkObjectPermission.py (original): the route set `user_id = current_user.id` and had no level checks at all -> ANY logged-in user could POST themselves `permissions = 4` (Owner) on ANY object = complete privilege escalation. Fixed by the whole rewrite.
2. networkObjectConnection.py delete_item: used `models.DBNetworkObject` instead of `models.DBNetworkObjectConnection`, so it looked up / deleted the wrong table. Fixed to DBNetworkObjectConnection.
3. networkObjectConnection.py create: `models.DBNetworkObjectConnection(**nOC.model_dump())` included `nO1` / `nO2`, which are not columns of that table -> would raise at runtime. Fixed with `model_dump(exclude={"nO1","nO2"})`.

4. networkObjectPermission: no uniqueness on (user_id, network_object_id), so a user could accumulate several conflicting permission rows. Mitigated by upserting in POST. (NOTE: a real fix should be a DB UNIQUE constraint on those two columns in models.py.)

—— Further problems Claude noticed but did NOT change (out of scope / need your decision) ——

#KI Claude detected problem why/what: a) auth.py: SECRET_KEY is hard-coded in the source and committed to git. It should come from an environment variable / config file and the leaked key be rotated. b) routers/userSettings.py references `current_user.user_settings_id`, but DBUser in models.py has no such column (and DBUserSettings is never auto-created on register). These routes will crash. Needs a model/relationship fix. c) routers/user.py register() is just `pass` -> no users can be created via the API, and there is no “create first admin” bootstrap (the TODO in the file). d) UserIn uses `hashed_password` but expects a plaintext password; passwords are never hashed/stored on register since register is unimplemented. e) Login/SNMP routers only check `nOP.user_id == current_user.id` (ownership of the permission row). That is correct for per-user logins, but they do NOT verify the user still has at least See(1) on the underlying object. Probably fine, flagging it.

========= USER SETTINGS + ADMIN BOOTSTRAP =========

Model: Claude (claude-opus-4-8) via Claude Code Date: 2026-06-16

User prompt (translated): “Can you fix userSettings: make every user get a userSettings on creation, and also make it so that on first start, when there is no DB yet, an admin user is created with password ‘admin’.”

Prompt-number -> code marker mapping (search `KI Claude <KI-N>`):

src/models.py + src/routers/userSettings.py - models.py: added a one-to-one `settings` relationship between DBUser and DBUserSettings (the user_settings_id column the route used never existed). - userSettings.py: look settings up by user_id via get_or_create_settings() (lazily creates them for legacy users); fixed UserSettingsOut to map the camelCase DB columns (showPorts/

showInterfaces) to the snake_case API fields via validation_alias; removed stray `from pip._internal...` import; fixed refresh-before-commit bug that discarded edits.

src/routers/user.py - Implemented register(): rejects duplicate usernames, hashes the password, creates the user AND a DBUserSettings row for them. - Renamed the misleading `hashed_password` input field to `password` (the value is plaintext and is hashed server-side).

src/main.py - create_default_admin(): on a fresh DB (no users) creates admin/admin with is_admin + canCreate and its settings. Registered the userSettings router (it was never included before).

Verified with a smoke test (throwaway sqlite DB): bootstrap admin created, settings defaults present, relationship works, login verifies admin/admin, and UserSettingsOut serializes showPorts->show_ports correctly.

This resolves earlier flagged problems (b), (c) and (d). Still open: (a) hard-coded SECRET_KEY, and the default admin password ‘admin’ should be changed after first login.

========= PUT PERSISTENCE BUG FIX =========

Model: Claude (claude-opus-4-8) via Claude Code Date: 2026-06-16

User prompt (translated): “Is anything still missing before a C# frontend UI can be attached?” -> Frontend will be a C# desktop app (WPF/WinForms/MAUI), so CORS is not required. User chose to fix only the refresh-before-commit bugs now.

src/routers/networkObject.py, networkObjectInterface.py, networkObjectConnection.py, snmpSettings.py, login.py - All PUT/edit handlers called self.db.refresh(obj) BEFORE self.db.commit(). refresh() reloads the row from the DB, throwing away the in-memory edits, so updates were silently lost. Swapped to commit() first, then refresh(). - (The POST/create handlers were already in the correct order.)

Verified with a throwaway-DB test: editing a NetworkObject’s name now persists across a fresh session. Compiles clean.

Noted but NOT changed (user deferred): no CORS middleware (fine for a desktop C# client using HttpClient, needed for any browser/Blazor frontend); PUT handlers `raise HTTPException(200)` instead of returning a success body; SECRET_KEY still hard-coded.

========= C# FRONTEND LINK (minimal backend additions) =========

Model: Claude (claude-opus-4-8) via Claude Code Date: 2026-06-16

Context: the C# client (RAT-Client) got a real IDatabaseConnection implementation (DatabaseConnection.cs) that talks to this backend over HTTP. The user asked NOT to change the backend unless necessary. Two things were necessary and are documented here. Marked in code with `KI Claude <KI-10>`.

src/routers/user.py - Added `can_create` to UserBase/UserOut/UserIn (validation_alias “canCreate” so it maps to the DBUser.canCreate column, populate_by_name so both names work). The client needs to know whether a user may create NetworkObjects (maps to RAT_Data.User.CanCreate / NetworkUser.CanCreate). - register() now persists `canCreate` from the request. - Added `GET /user/` (list all users, auth required). The client needs it to resolve the user_id stored in a NetworkObjectPermission back to a username (Access Control tab) and to implement IDatabaseConnection.GetAllUsers().

```
     src/routers/networkObjectInterface.py
      #KI Claude detected problem why/what:
NetworkObjectInterfaceOut.
      network_object_connection_id was typed as a non-
optional `int`, but the column
      is NULL for any interface not attached to a
connection. GET /networkObjectInterface/
      then crashed with a ResponseValidationError (None is
not an int), which blocked
      the C# client from loading the topology. Changed the
type to `int | None`.
```

Verified against a throwaway DB with the live server: form login -> JWT; /user/me and /user/ return can_create; NetworkObject create/list; the creator’s permission row is Owner(4); per-device login create/list; PUT /user/settings/ returns 200; and after the interface fix, interface + connection create/list serialize correctly. All response shapes match the client’s DTOs.

Still open (unchanged): hard-coded SECRET_KEY; default admin password ‘admin’; no CORS (not needed for the desktop HttpClient client).

========= CASCADE DELETE FOR NETWORKOBJECT =========

Model: Claude (claude-opus-4-8) via Claude Code Date: 2026-06-16

Context: second pass on the C# client (RAT-Client prompt 14) made it edit/delete interfaces, connections, logins and permissions through the API. While testing the delete path, the delete route turned out to leave dangling rows.

src/routers/networkObject.py #KI Claude detected problem why/what: delete_item only removed the NetworkObject and its permission rows. Its interfaces (and the connections those interfaces used, plus the logins / snmp settings on the permission rows) stayed behind as orphans, and on the next graph load the client then referenced interfaces whose object no longer exists. delete_item now cascade-deletes: interfaces of the object -> the connections those interfaces used -> logins + snmp settings on the object’s permission rows -> the permission rows -> the object itself.

Verified against a throwaway DB with the live server: created two objects with one interface each and a connection between them, granted another user See on one object, then deleted that object. Its interface and the connection were gone afterwards while the other object’s interface remained. (Minor, left as-is: the surviving interface’s network_object_connection_id still points at the deleted connection; harmless because the client only rebuilds a connection when BOTH endpoints reference it.)

========= EDIT USER ENDPOINT =========

Model: Claude (claude-opus-4-8) via Claude Code Date: 2026-06-17

Context: the C# client needed a way to edit users (change username / password / admin / can_create). The API only had register (create) — EditUser had no endpoint.

src/routers/user.py - Added a `UserEdit` model (all fields optional: username / password / is_admin / can_create) and `PUT /user/{id}`. Authorization: * a global admin may edit ANY user and ANY field * a normal user may edit ONLY themselves, and only username + password (is_admin / can_create from the payload are ignored for a non-admin self-edit; editing any other user returns 403) An empty/ missing password leaves the current one unchanged; the password is hashed server-side; username uniqueness is enforced (409 on clash).

Verified against a throwaway DB: admin renames + promotes another user (200); a normal user self-edits name+password (200, re-login with the new password works); a normal user editing someone else is 403; a normal user self-edit trying to set is_admin/can_create is accepted but ignored (the row stays non-admin / can_create False).

This is what the client's IDatabaseConnection.EditUser (PUT /user/{id}) talks to, used by the admin "Manage Users" Edit button and the per-user "Edit my account" in Settings.

========= PASSWORD POLICY =========

Model: Claude (claude-opus-4-8) via Claude Code Date: 2026-06-18

Context: passwords were stored without any strength check (frontend or backend). The client now validates client-side (RAT_Logic.PasswordPolicy) and the backend must enforce the same so a weak password can never reach the database.

src/routers/user.py - Added validate_password(password): enforces the policy * at least 8 characters * at least one letter * at least one digit Raises HTTP 400 with a clear message otherwise. - Called from register() (new user) and from edit_user() when a new password is supplied (an empty password on edit still means "leave unchanged", so it is not checked then).

Marked in code with `KI Claude <KI-13>`. Mirrors the C# client's RAT_Logic.PasswordPolicy (same rules) so validation is consistent on both ends.

========= DELETE USER ENDPOINT =========

Model: Claude (claude-opus-4-8) via Claude Code Date: 2026-06-18

Context: a global admin needed to be able to delete users (the API only had register + edit).

src/routers/user.py - Added `DELETE /user/{id}` (global-admin only). An admin may not delete their own account (so the system can't be left with no admin by accident; 400). Cleans up everything that hangs off the user before deleting them: their UserSettings, their NetworkObjectPermission rows, and the Login / SNMPSettings rows stored against those permission rows — otherwise those would dangle and the C# graph load would break. - Added `from starlette import status` for the 200 result.

This is what the client's IDatabaseConnection.DeleteUser (DELETE /user/{id}) now talks to, used by the admin "Manage Users" per-row Delete button. (Editing users — name/password/admin/can_create — was already covered by PUT /user/{id}, KI-12; password changes are validated by KI-13.)

========= REQUIREMENTS PASS: LOGGING, AGGREGATION, PAGINATION/FILTER, CLOUD-DB, STATUS CODES =========

Model: Claude (claude-opus-4-8) via Claude Code Date: 2026-06-21

Overall user prompt (translated/summarized): "Make sure these required points are met and that we only raise REAL error messages: at least one aggregation endpoint (GROUP BY + COUNT/SUM/AVG); HTTP status codes set correctly (200/201/400/404/ 409/401); parametrized queries everywhere (SQL-injection safe); a continuously kept project diary (who/when/what) in the same style as the frontend, built from the git commits. Also check the mandatory logging (Python logging module, INFO for requests, ERROR for errors, written to BOTH the console and a

file api.log) and the four extra tasks: extended filtering (query params), pagination (limit/offset), a cloud database, and extended aggregation/statistics. Mark AI usage as before in AI_usage.md and in the code with `KI start / KI end` plus which prompt."

Prompt-number -> code marker mapping (search `KI Claude <KI-N>`):

src/main.py - Logging was writing to "rat.tail" and never logged errors. Changed the FileHandler to "api.log" (the required filename) so logs go to BOTH the console and api.log. - Added a global `@app.exception_handler(Exception)` that ERROR-logs any unhandled error (with traceback) and returns a clean 500 (no stack trace / DB internals leaked). - The validation handler now also logs validation failures as warnings. - The existing http middleware keeps INFO-logging every request (method, path, status).

src/routers/statistics.py (NEW FILE) + registered in src/main.py - New read-only `/statistics` router with the required GROUP BY + COUNT/SUM/AVG: * GET /statistics/summary -> COUNT of objects/interfaces/connections plus AVG/SUM/MAX/MIN over connection speed * GET /statistics/objects-by-type -> GROUP BY NetworkObject.type, COUNT * GET /statistics/connections-by-type-> GROUP BY connection.type, COUNT + AVG + SUM speed * GET /statistics/interfaces-per-object-> GROUP BY object, COUNT of its interfaces - Same visibility rules as the other routers: a normal user only sees stats over objects they have See(1)+ on; a global admin sees everything (so stats can't leak hidden objects). - All queries use the SQLAlchemy ORM with bound parameters (SQL-injection safe).

src/routers/networkObject.py, networkObjectInterface.py, networkObjectConnection.py, user.py - Added optional query parameters to the GET list endpoints (every param has a default, so the existing C# client keeps working unchanged): * networkObject: name (ILIKE search), type (filter), sort_by, order, limit, offset * networkObjectInterface: network_object_id, name, is_up, limit, offset * networkObjectConnection: name, type, min_speed, sort_by, order, limit, offset * user: username (search), is_admin, limit, offset - Filtering uses bound LIKE/equality parameters; sorting uses a WHITELIST of allowed columns (so an arbitrary sort string can never reach the SQL). Pagination via offset()/limit().

src/database.py + src/.env.example (NEW FILE) - The DB URL now comes from the DATABASE_URL environment variable (read from a real env var or a gitignored `.env` next to database.py), with a fallback to the local SQLite file. This

lets the API be pointed at a Cloud database (Supabase / Railway / PlanetScale) WITHOUT a code change — only set DATABASE_URL and install the matching driver (see .env.example). - check_same_thread is only passed for SQLite URLs (it is a SQLite-only argument). - NOTE: the actual cloud connection string + driver still have to be filled in by the team; the code side is done.

src/routers/networkObject.py, networkObjectInterface.py, networkObjectConnection.py, networkObjectPermission.py, snmpSettings.py, login.py, userSettings.py, user.py - “Only real error messages”: every successful PUT/DELETE used to `raise HTTPException(200)` — raising an exception for a SUCCESS is wrong (exceptions are for errors, and it muddied the status semantics). They now simply `return {"detail": "..."}` and the PUT decorators were changed from status_code=201 to 200 (an update is 200, not 201) so the observed status the C# client sees stays 200 as before. #KI Claude detected problem why/what: - PUT /networkObjectPermission/{id} called self.db.refresh() BEFORE self.db.commit(), so the `permissions` change was reloaded away and the update silently did nothing (the same refresh-before-commit bug fixed elsewhere as KI-9, but missed in this router). Fixed: commit() first, then refresh().

—— Status-code review (no change needed, confirmed correct) —— 200 = successful GET / PUT / DELETE (now via a normal response body, KI-19) 201 = successful POST/create (create_* handlers) 400 = validation error (global handler) / password policy (KI-13) / bad permission edit (KI-5) 401 = wrong credentials on login and invalid/expired JWT (user.py login, auth.get_current_user) 404 = unknown id (BaseAPI.get_or_404) and hidden objects (permissions.require_permission) 409 = duplicate username (register/edit_user) and duplicate NetworkObject name (create)

—— SQL-injection review (no change needed, confirmed safe) —— All DB access goes through the SQLAlchemy ORM / Query API, which always uses bound parameters. The new filters (KI-17) and statistics (KI-16) use ILIKE/equality on column objects (bound) and a whitelist for sort columns — no string-concatenated SQL anywhere.

—— Project diary —— Created doc/markdown/Projekttagebuch.md (who/when/what), derived from the git commits and written in the frontend’s ADDED/FIXED/CHANGED style. Mapping: sumpfel = Christof, Pir4t3141 = Tobias.

Verified with a live smoke test against a throwaway SQLite DB (TestClient): bootstrap admin login (200), NetworkObject create (201), duplicate name (409),

filtered+sorted list with pagination, PUT edit returns 200 with a body AND the change persists, the statistics endpoints return the GROUP BY / COUNT / AVG/ SUM/MIN/MAX results, missing token -> 401, unknown id on PUT -> 404, and api.log is written. Separately verified that PUT /networkObjectPermission/{id} now persists the permission-level change (the refresh-before-commit fix). All modules also compile cleanly (`python -m py_compile`).

========= SUBMISSION STRUCTURE: uvicorn src.main:app + init_db.py =========

Model: Claude (claude-opus-4-8) via Claude Code Date: 2026-06-21

User prompt (summarized): the submission requires the API to be startable via `uvicorn src.main:app`, an init_db.py at the project root, a complete requirements.txt, and a fixed doc layout (doc/DBI_Dokumentation_.pdf + doc/erm.drawio). Make sure all of that holds; init_db.py should write directly into the DB file and seed dummy test data.

src/main.py, src/**init**.py, src/database.py, init_db.py (NEW), requirements.txt (NEW root) - Problem: the modules import each other flat (`from database import ...`), which only resolves when src/ is the cwd. `uvicorn src.main:app` (from the project root) therefore crashed with ModuleNotFoundError. Fix: added src/**init**.py and, at the top of main.py, insert this file's directory (src/) into sys.path before the flat imports. Now both `uvicorn src.main:app` (root) and `uvicorn main:app` (src/) work. - database.py: the default SQLite path was relative (`./RATBASE.db`), so the server (run from root) and init_db.py disagreed on the file. Anchored the default to an ABSOLUTE src/RATBASE.db. Likewise anchored api.log to src/api.log (was relative to cwd). - init_db.py (NEW, project root): creates all tables and, on a fresh DB, seeds dummy test data (admin/admin + bob, two NetworkObjects, two interfaces, a connection, four permission rows, a login and SNMP settings). Writes directly into src/RATBASE.db (override via DATABASE_URL). Idempotent: skips seeding if users already exist. - requirements.txt (NEW, project root): `-r src/requirements.txt` so a root-level `pip install -r requirements.txt` works and stays in sync with the pinned list.

Verified: `uvicorn src.main:app` from the project root boots (/, /docs -> 200), admin login works, and /statistics/summary reflects the data init_db.py seeded (2 objects / 2 interfaces / 1 connection, AVG/SUM/MIN/MAX over speed). A fresh venv with ONLY requirements.txt installed imports the whole app.

Docs placed for submission: doc/DBI_Dokumentation_RAT-Backend.pdf (the consolidated Dokumentation.md) and doc/erm.drawio (copy of doc/assets/drawio/DB_Sketches_v2.drawio).